



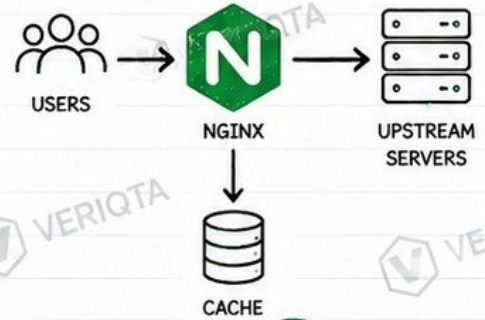
WHAT IS NGINX?

High-performance web server, reverse proxy, load balancer, and API gateway.

- ✓ Reverse Proxy
- ✓ Load Balancing
- ✓ TLS Termination
- ✓ Static Content
- ✓ High Concurrency



PRODUCTION TRAFFIC FLOW



NGINX PRODUCTION HANDBOOK

BUILT FOR

- ✓ DevOps Engineers
- ✓ Site Reliability Engineers
- ✓ Platform Engineers
- ✓ System Administrators
- ✓ Backend Engineers

A PRACTICAL, PRODUCTION-READY GUIDE

Building, Securing, Scaling, and Troubleshooting
NGINX in Real-World Production Environments



SECURITY

Hardening, TLS, and protection best practices.



LOAD BALANCING

Distribute traffic efficiently and reliably.



PERFORMANCE

Optimize for speed, concurrency, and resource efficiency.



OBSERVABILITY

Logging, metrics, and monitoring in production.



TROUBLESHOOTING

Real incidents, errors, and proven debugging steps.



PRODUCTION FOCUSED | PRACTICAL | RELIABLE | BATTLE TESTED



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

NGINX IN PRODUCTION



WHAT NGINX DOES IN MODERN INFRASTRUCTURE



WEB SERVER

Serves websites and web applications.



REVERSE PROXY

Forwards client requests to backend servers securely.



LOAD BALANCER

Distributes traffic across multiple backend servers.



API GATEWAY USE CASES

Handles routing, rate limiting, authentication, and more.



TLS TERMINATION

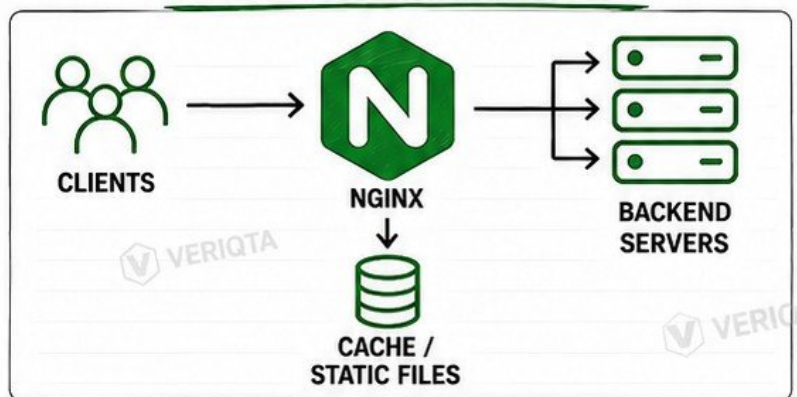
Offloads SSL/TLS encryption and improves performance.



STATIC CONTENT DELIVERY

Delivers static files fast and efficiently with low resources.

TYPICAL PRODUCTION REQUEST FLOW



NGINX VERSUS APPLICATION SERVERS

NGINX (EDGE / INFRASTRUCTURE)	APPLICATION SERVERS (BUSINESS LOGIC)
✓ HIGH PERFORMANCE ✓ LOW MEMORY USAGE ✓ HANDLES THOUSANDS OF CONNECTIONS ✓ EXCELLENT FOR STATIC CONTENT ✓ SSL OFFLOADING ✓ LOAD BALANCING ✓ REVERSE PROXY ✓ LOWER INFRASTRUCTURE COST	✓ PROCESS BUSINESS LOGIC ✓ GENERATE DYNAMIC RESPONSES ✓ DATABASE OPERATIONS ✓ USER SESSIONS ✓ APPLICATION-SPECIFIC FUNCTIONS ✓ MORE RESOURCE INTENSIVE ✓ NOT IDEAL AS EDGE PROXY

PRODUCTION ARCHITECTURE OVERVIEW



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

INSTALLING AND STRUCTURING NGINX



1. INSTALLING NGINX ON LINUX

```
# Ubuntu / Debian
sudo apt update
sudo apt install nginx -y

# RHEL / CentOS / Rocky / AlmaLinux
sudo dnf install nginx -y

# Start and enable NGINX
sudo systemctl enable nginx
sudo systemctl start nginx
```

2. PACKAGE REPOSITORIES

- Ubuntu / Debian: Official repositories
- RHEL / CentOS / Rocky / AlmaLinux: EPEL repository
- For the latest version, use the official NGINX repository

https://nginx.org/en/linux_packages.html

3. SERVICE MANAGEMENT WITH SYSTEMD

```
# Check status
sudo systemctl status nginx
# Start
sudo systemctl start nginx
# Stop
sudo systemctl stop nginx
# Restart
sudo systemctl restart nginx
# Reload (graceful)
sudo systemctl reload nginx
# Enable on boot
sudo systemctl enable nginx
```

4. NGINX -T (CONFIG TEST)

Always test configuration before reloading.

```
sudo nginx -t
```

If the configuration is valid, you will see:

```
nginx: the configuration file /etc/nginx/nginx.conf
syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test
is successful
```

5. IMPORTANT DIRECTORIES

DIRECTORY / FILE	PURPOSE	EXAMPLE / NOTES
 /etc/nginx/nginx.conf	Main configuration file	Top-level configuration
 /etc/nginx/conf.d/	Additional config files	*.conf files loaded here
 /etc/nginx/sites-available/	Virtual host configs (not enabled)	Place new site configs here
 /etc/nginx/sites-enabled/	Enabled virtual hosts	Symlinked from sites-available
 /var/www/html/	Default document root	Serves static files
 /var/log/nginx/	Log directories	access.log, error.log
 /run/nginx.pid	PID file	Process ID of master process

6. RELOAD VERSUS RESTART

RELOAD (PREFERRED)

- ✓ Applies configuration changes without dropping connections.
- ✓ Uses a graceful process management.

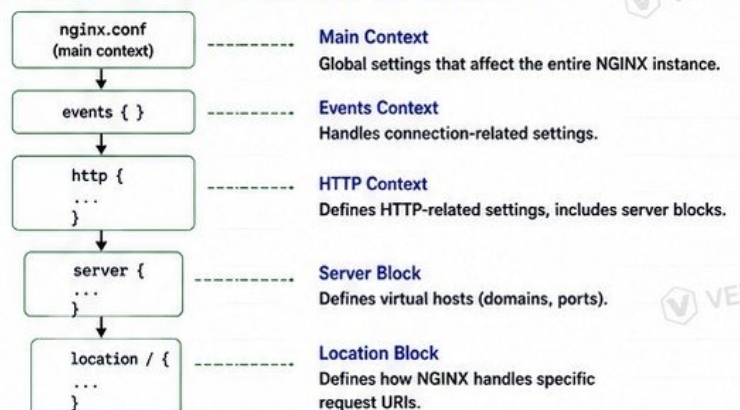
```
sudo systemctl reload nginx
```

RESTART

- ✗ Stops and starts NGINX.
- ✗ All existing connections are dropped.

```
sudo systemctl restart nginx
```

7. UNDERSTANDING THE CONFIGURATION HIERARCHY



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta

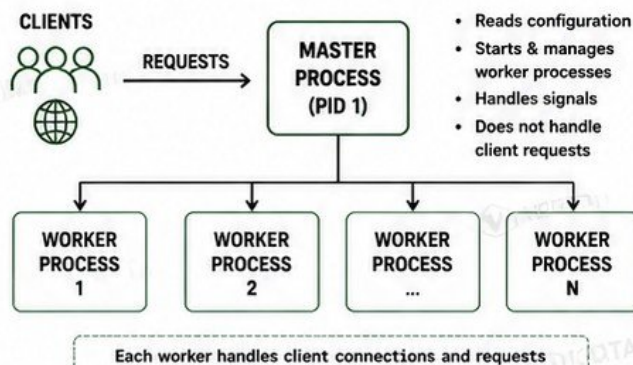


Telegram
veriqta

NGINX PROCESSES AND WORKER ARCHITECTURE



1. NGINX PROCESS MODEL



2. EVENT-DRIVEN ARCHITECTURE



3. WORKER CONNECTIONS

- Each worker can handle many simultaneous connections.
- Total connections = $\text{worker_processes} \times \text{worker_connections}$
- Connections include clients, upstream backends, keepalive, etc.

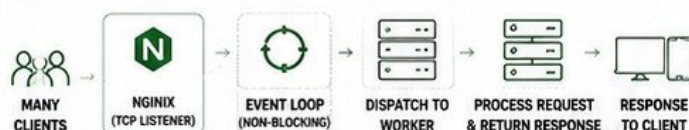
Example:
`worker_processes 4;`
`worker_connections 10240;`
Max theoretical connections
 $= 4 \times 10240 = 40,960$
(per worker type, not exact client connections)

4. CORE WORKER DIRECTIVES

DIRECTIVE	DESCRIPTION	EXAMPLE
<code>worker_processes</code>	Number of worker processes.	<code>worker_processes auto;</code>
<code>worker_connections</code>	Max connections per worker.	<code>worker_connections 10240;</code>
<code>multi_accept</code>	Accept as many connections as possible at once.	<code>multi_accept on;</code>
<code>use</code>	Event module to use.	<code>use epoll;</code>

TIP: `worker_processes auto;` is recommended in production.

10. HOW NGINX HANDLES CONCURRENT TRAFFIC



5. CONNECTION CAPACITY

- Connection capacity depends on:
 - `worker_processes`
 - `worker_connections`
 - File descriptor limits
 - System memory
- Always plan capacity based on real workload and backends.

6. FILE DESCRIPTOR LIMITS

- NGINX uses file descriptors for every connection. If limits are too low, you will see errors.

```
# Check current limit
ulimit -n

# Increase limit (temporary)
ulimit -n 65535

# Set permanently (example for systemd)
# In /etc/security/limits.conf
* soft nofile 65535
* hard nofile 65535
```

Ensure the limit is higher than:
 $\text{worker_processes} \times \text{worker_connections} \times 1.5$

7. EVENT MODULES

MODULE	DESCRIPTION	BEST FOR
epoll	High-performance event notification on Linux.	Linux (recommended)
kqueue	Event notification on BSD/macOS.	BSD / macOS
select	Portable but less efficient.	Legacy systems
poll	Portable, better than select.	Legacy systems

8. EPOLL ON LINUX

- Epoll is scalable and efficient for thousands of connections.
- Recommended for all modern Linux production environments.

In `nginx.conf`
`use epoll;`

9. GRACEFUL RELOADS

- Sends new config to master process.
- Master starts new workers with new config.
- Old workers finish existing connections and exit gracefully.
- Zero downtime reload.

```
# Test configuration first
nginx -t

# Reload gracefully
systemctl reload nginx
```

11. PRODUCTION WORKER CONFIGURATION (EXAMPLE)

```
worker_processes auto;
worker_connections 10240;
use epoll;
multi_accept on;
worker_rlimit_nofile 65535;

# Optional tuning
keepalive_timeout 65;
keepalive_requests 10000;
```

NOTES

- ✓ `auto` = matches CPU cores
- ✓ Use `epoll` on Linux
- ✓ Increase file descriptor limits
- ✓ Test and monitor capacity
- ✓ Tune based on real load



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

SERVER BLOCKS AND REQUEST MATCHING



1. HTTP CONTEXT

The http context is the top-level block for most NGINX configuration.

```
http {
    include      mime.types;
    default_type application/octet-stream;
    sendfile     on;
    keepalive_timeout 65;

    # Server blocks live here
    server { ... }
    server { ... }
}
```

2. SERVER BLOCKS

A server block defines how NGINX handles requests for a host and port.

```
server {
    listen      80;
    server_name example.com www.example.com;

    location / {
        root /var/www/html;
    }
}
```

3. LISTEN

Defines the IP address and port the server listens on.

- listen 80;
- listen 443 ssl;
- listen 127.0.0.1:8080;
- listen [::]:80; (IPv6)

TIP You can have multiple listen directives in one server block.

4. SERVER_NAME

Matches the Host header from the client.

- server_name example.com;
- server_name example.com www.example.com;
- server_name *.example.com;
- server_name _; (catch-all)

5. DEFAULT SERVER

If no server_name matches, NGINX uses the default server.

- The first server block listening on a port becomes the default.
- Or explicitly mark it:

```
listen 80 default_server;
listen 443 ssl default_server;
```

6. MULTIPLE DOMAINS

One server block can handle many domains.

```
server {
    listen      80;
    server_name example.com www.example.com
               api.example.com;
    root /var/www/example;
}
```

7. LOCATION BLOCKS

Location blocks define how NGINX handles different parts of a URL.

```
server {
    listen 80;
    server_name example.com;

    location / {
        root /var/www/html;
        index index.html;
    }

    location /images/ {
        alias /data/images/;
    }

    location = /health {
        return 200 "OK";
    }
}
```

8. LOCATION MATCHING TYPES

TYPE	SYNTAX	EXAMPLE	DESCRIPTION
Exact Match	=	location = /health { ... }	Matches the exact URI. Highest priority.
Prefix Match	(no modifier)	location /app/ { ... }	Matches URI prefix. Longest prefix wins.
Regular Expression	~	location ~ /\.php\$ { ... }	Case-sensitive regex match.
Case-Insensitive	~*	location ~* /\.jpg\$ { ... }	Case-insensitive regex match.
Prefix (no slash)	(no modifier)	location /about { ... }	Matches prefix (e.g., /about, /about/team)

10. ROOT VERSUS ALIAS

ROOT	ALIAS
Combines the location path with the root path.	Replaces the location path with the alias path.
<pre>location /images/ { root /var/www/html; }</pre>	<pre>location /images/ { alias /data/images/; }</pre>
Request: /images/logo.png Result: /var/www/html/images/logo.png	Request: /images/logo.png Result: /data/images/logo.png

9. LOCATION MATCHING PRECEDENCE (HIGH → LOW)

- 1 Exact Match (=)
- 2 Longest Prefix Match.
- 3 Regex location (~, ~*) in order of appearance
- 4 Shorter Prefix Matches
- 5 Default / (fallback)

NOTE

Regex locations are only checked if no prefix match is found (except when using ~* modifier).

11. COMMON ROUTING MISTAKES

- ⊗ Using root when alias is required (or vice versa).
- ⊗ Forgetting trailing slashes.
- ⊗ Overlapping location blocks causing unexpected matches.
- ⊗ Regex locations placed before specific prefix locations.
- ⊗ Multiple server blocks listening on the same port without understanding default server behavior.
- ⊗ Forgetting to reload after changes.

12. TESTING REQUEST MATCHING

Test what location block will handle a request.

Check configuration

```
sudo nginx -t
```

Test with curl

```
curl -I -H "Host: example.com" \
http://localhost/about/team
```

Debug logging (optional)

```
error_log /var/log/nginx/
error.log debug;
```

TIP Use browser DevTools → Network → Headers to inspect request/response details.

QUICK REFERENCE: SERVER BLOCK ESSENTIALS

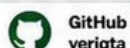
DIRECTIVE	PURPOSE	EXAMPLE
listen	Defines IP and port to listen on.	listen 80;
server_name	Defines domains the server block handles.	server_name example.com www.example.com;
root	Sets the document root for the location.	root /var/www/html;
alias	Maps a location to a different directory.	alias /data/images/;
location	Defines rules for specific URI paths.	location /api/ { proxy_pass http://backend; }

BEST PRACTICE

- ✓ Keep one responsibility per server block.
- ✓ Use descriptive server_name values.
- ✓ Prefer HTTPS (listen 443 ssl).
- ✓ Test configuration before reloading.
- ✓ Understand location matching to avoid surprises.



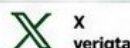
YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



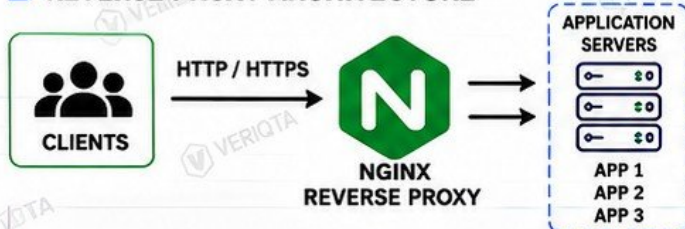
Instagram
veriqta



Telegram
veriqta

NGINX AS A REVERSE PROXY

REVERSE PROXY ARCHITECTURE



- ✓ NGINX ACCEPTS CLIENT REQUESTS AND FORWARDS THEM TO BACKEND APPLICATION SERVERS. THE RESPONSE FROM THE SERVERS IS RETURNED TO THE CLIENT.

PROXY_PASS

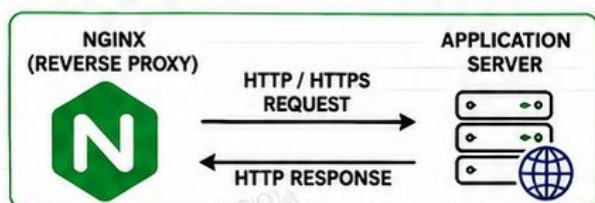
```
location / {
    proxy_pass http://app_backend;
}

upstream app_backend {
    server 10.0.0.10:8080;
    server 10.0.0.11:8080;
    server 10.0.0.12:8080;
}
```



PROXY_PASS DEFINES WHERE NGINX SENDS THE REQUEST. IT CAN POINT TO A SINGLE SERVER OR AN UPSTREAM GROUP.

PROXYING TO APPLICATION SERVERS



- ✓ NGINX HIDES THE BACKEND SERVERS FROM CLIENTS AND HANDLES CONNECTION MANAGEMENT, SECURITY, AND LOAD DISTRIBUTION.

REQUEST HEADERS

HEADER	PURPOSE
Host	Preserves original host requested by client.
X-Real-IP	Sends the actual client IP to backend.
X-Forwarded-For	Sends the client IP chain through proxies.
X-Forwarded-Proto	Sends the original protocol (http or https).

EXAMPLE: PASSING CLIENT INFORMATION

```
location / {
    proxy_pass http://app_backend;

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

- ✓ THIS ENSURES THE BACKEND APPLICATION KNOWS THE ORIGINAL CLIENT IP, HOST, AND PROTOCOL.

PROXY HTTP VERSIONS

SETTING	DESCRIPTION
proxy_http_version 1.1;	Required for keepalive, WebSockets, and HTTP/1.1 features.
proxy_set_header Connection "";	Clears the Connection header for proper keepalive.
proxy_set_header Upgrade \$http_upgrade;	Enables protocol upgrades (WebSocket support).
proxy_set_header Connection \$connection_upgrade;	Used with WebSocket proxying.

REQUEST BUFFERING

DIRECTIVE	DESCRIPTION
proxy_request_buffering on;	Buffers the client request body before sending to backend.
proxy_request_buffering off;	Streams the request body to backend without buffering.

- ✓ DISABLE BUFFERING FOR LARGE UPLOADS OR STREAMING TO REDUCE LATENCY AND MEMORY USAGE.

PROXY TIMEOUTS

DIRECTIVE	DESCRIPTION
proxy_connect_timeout 5s;	Time allowed to connect to upstream.
proxy_send_timeout 60s;	Time allowed to send request to upstream.
proxy_read_timeout 60s;	Time to wait for response from upstream.
send_timeout 60s;	Time allowed to send response to client.

- ⚠ INCORRECT TIMEOUTS CAN CAUSE 502 BAD GATEWAY OR 504 GATEWAY TIMEOUT ERRORS.

RESPONSE BUFFERING

DIRECTIVE	DESCRIPTION
proxy_buffering on;	Buffers backend responses before sending to client.
proxy_buffering off;	Streams backend responses directly to client.

- ✓ DISABLE BUFFERING FOR REAL-TIME DATA, LARGE DOWNLOADS, OR LOW LATENCY STREAMS.

TROUBLESHOOTING UPSTREAM COMMUNICATION

- ✓ CHECK BACKEND SERVER STATUS AND AVAILABILITY.
- ✓ VERIFY NETWORK CONNECTIVITY BETWEEN NGINX AND BACKEND.
- ✓ CHECK NGINX ERROR LOG FOR UPSTREAM ERRORS.
- ✓ CONFIRM CORRECT proxy_pass AND UPSTREAM CONFIGURATION.
- ✓ CHECK TIMEOUT SETTINGS.
- ✓ VERIFY DNS RESOLUTION IF USING DOMAIN NAMES.
- ✓ USE curl OR telnet TO TEST BACKEND DIRECTLY.
- ✓ MONITOR UPSTREAM RESPONSE TIMES AND FAILURES.

⚙️ GOOD LOGS + CORRECT TIMEOUTS + HEALTHY BACKENDS = RELIABLE REVERSE PROXY SETUP.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta

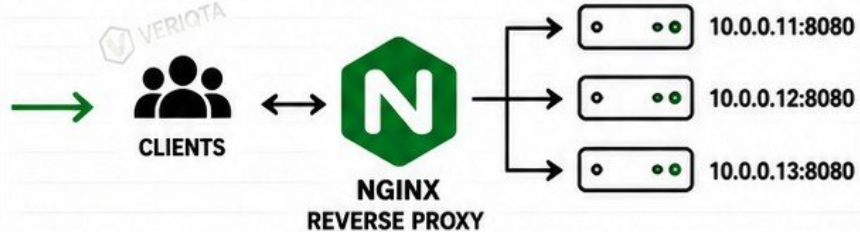


Telegram
veriqta

PRODUCTION LOAD BALANCING

UPSTREAM BLOCKS

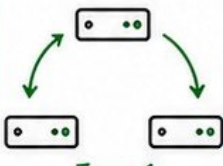
```
upstream app_backend {
    server 10.0.0.11:8080;
    server 10.0.0.12:8080;
    server 10.0.0.13:8080;
}
```



**MULTIPLE
BACKEND
SERVERS**

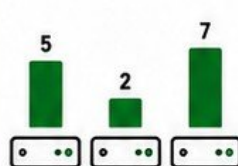
LOAD BALANCING METHODS

ROUND ROBIN



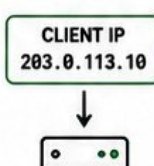
Requests are distributed sequentially to each server in order.

LEAST CONNECTIONS



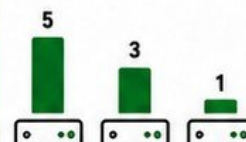
Requests are sent to the server with the least active connections.

IP HASH



A client IP is mapped to a specific server using a hash function.

WEIGHTED LOAD BALANCING



Servers receive traffic based on their assigned weights.

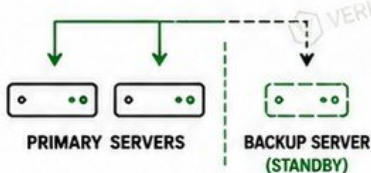
BACKEND WEIGHTS

WEIGHTS

server1 weight=5
server2 weight=3
server3 weight=1

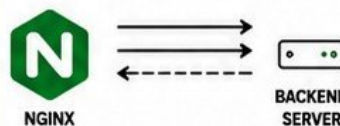
Higher weight servers handle more requests proportionally.

BACKUP SERVERS



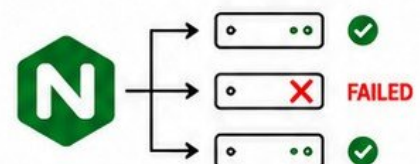
Backup servers are used only when all primary servers are unavailable.

CONNECTION REUSE (KEEPALIVE CONNECTIONS)



Keepalive reuses existing TCP connections to reduce latency and improve performance.

BACKEND FAILURE HANDLING



NGINX automatically stops sending traffic to failed servers and routes to healthy ones.

LOAD-BALANCING ARCHITECTURE



NGINX SITS AT THE FRONT, DISTRIBUTING TRAFFIC, TERMINATING TLS, AND PROTECTING BACKEND SERVICES.

CHOOSING AN ALGORITHM

ALGORITHM	BEST FOR	CONSIDERATIONS
Round Robin	General purpose, even workloads	Simple and effective when servers are similar.
Least Connections	Long-lived connections, APIs	Better for uneven connection durations.
IP Hash	Session persistence without cookies	Not ideal if client IPs are behind NAT.
Weighted Load Balancing	Servers with different capacities	Distribute more traffic to stronger servers.
Backup Servers	High availability	Used only when all primary servers fail.



CHOOSE THE RIGHT METHOD BASED ON YOUR APPLICATION PATTERN, SERVER CAPACITY, SESSION REQUIREMENTS, AND FAILURE TOLERANCE.



MONITOR, TEST, AND ADJUST YOUR STRATEGY AS YOUR TRAFFIC GROWS.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta

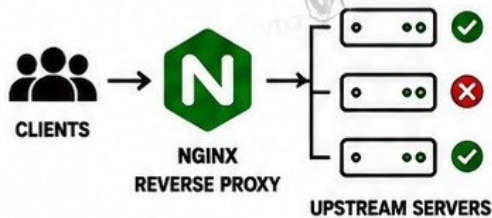


Telegram
veriqta

HEALTH CHECKS AND UPSTREAM FAILURE HANDLING

PASSIVE HEALTH CHECKS

NGINX DOES NOT SEND ACTIVE PROBES. IT MONITORS REAL TRAFFIC AND MARKS BACKENDS AS FAILED BASED ON ERRORS OR TIMEOUTS.

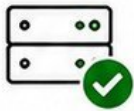


MAX_FAILS
NUMBER OF FAILED ATTEMPTS BEFORE A SERVER IS MARKED AS UNAVAILABLE.

FAIL_TIMEOUT
TIME DURATION TO KEEP A FAILED SERVER OUT OF THE ROTATION.

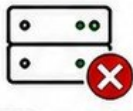
```
upstream app_backend {
    server 10.0.0.11:8080 max_fails=3 fail_timeout=30s;
    server 10.0.0.12:8080 max_fails=3 fail_timeout=30s;
    server 10.0.0.13:8080 max_fails=3 fail_timeout=30s;
}
```

UPSTREAM AVAILABILITY



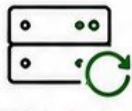
ONLY AVAILABLE SERVERS RECEIVE TRAFFIC. FAILED SERVERS ARE SKIPPED.

FAILED BACKEND DETECTION



AFTER `max_fails` IS REACHED WITHIN `fail_timeout`, NGINX STOPS SENDING TRAFFIC TO THAT SERVER.

BACKEND RECOVERY



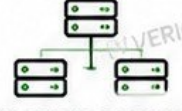
AFTER `fail_timeout` EXPIRES, THE SERVER IS CONSIDERED RECOVERED AND IS TRIED AGAIN WITH A NEW REQUEST.

AVOIDING CASCADING FAILURES



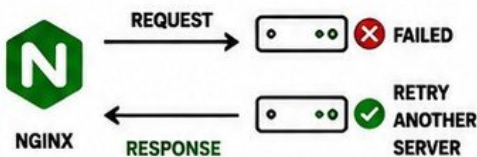
NGINX STOPS SENDING TRAFFIC TO UNHEALTHY SERVERS TO PREVENT OVERLOADING THEM AND PROTECT OTHER DEPENDENCIES.

DESIGNING RESILIENT UPSTREAM POOLS



USE MULTIPLE SERVERS, APPROPRIATE TIMEOUTS, AND RETRY RULES TO ENSURE HIGH AVAILABILITY AND FAULT TOLERANCE.

RETRY BEHAVIOR



PROXY_NEXT_UPSTREAM

DIRECTS NGINX TO RETRY THE REQUEST ON ANOTHER SERVER WHEN CERTAIN CONDITIONS ARE MET.

```
proxy_next_upstream error timeout http_502
                    http_503 http_504;
proxy_next_upstream_tries 3;
proxy_next_upstream_timeout 10s;
```

RETRY CONDITIONS

- ✓ ERROR
- ✓ TIMEOUT
- ✓ HTTP_502
- ✓ HTTP_503
- ✓ HTTP_504
- ✓ INVALID_HEADER
- ✓ NON_IDEMPOTENT (CARE!)

RETRY RISKS FOR NON-IDEMPOTENT REQUESTS



NON-IDEMPOTENT REQUESTS (POST, PATCH, DELETE) MAY CAUSE DUPLICATE ACTIONS IF RETRIED AFTER THE BACKEND ACTUALLY PROCESSED THE REQUEST BUT NGINX DID NOT RECEIVE THE RESPONSE.

EXAMPLES

- CHARGING A CREDIT CARD
- CREATING ORDERS
- TRANSFERRING MONEY
- SENDING EMAILS

USE RETRY CAREFULLY FOR NON-IDEMPOTENT REQUESTS. !

RECOMMENDED PRODUCTION SETTINGS EXAMPLE

```
upstream app_backend {
    server 10.0.0.11:8080 max_fails=3 fail_timeout=30s;
    server 10.0.0.12:8080 max_fails=3 fail_timeout=30s slow_start=30s;
    server 10.0.0.13:8080 max_fails=3 fail_timeout=30s backup;
    keepalive 32;
}

server {
    location / {
        proxy_pass http://app_backend;
        proxy_next_upstream error timeout http_502 http_503 http_504;
        proxy_next_upstream_tries 3;
        proxy_next_upstream_timeout 10s;
    }
}
```

BEST PRACTICES

- ✓ SET APPROPRIATE `max_fails` AND `fail_timeout`.
- ✓ USE RETRY ONLY FOR SAFE CONDITIONS.
- ✓ LIMIT `proxy_next_upstream_tries`.
- ✓ MONITOR ERROR RATES AND LATENCIES.
- ✓ USE MULTIPLE BACKENDS ACROSS FAILURE DOMAINS.
- ✓ TEST FAILURE SCENARIOS REGULARLY.
- ✓ KEEP TIMEOUTS REALISTIC.
- ✓ USE CONNECTION REUSE (KEEPALIVE).



A HEALTHY UPSTREAM POOL IS THE FOUNDATION OF A RELIABLE NGINX DEPLOYMENT.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



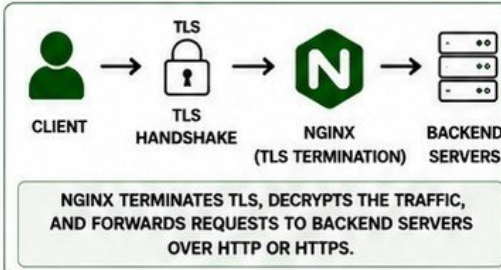
Instagram
veriqta



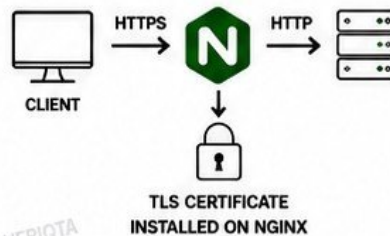
Telegram
veriqta

TLS AND HTTPS IN PRODUCTION

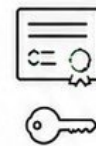
1. HTTPS REQUEST FLOW



2. TLS TERMINATION



3. CERTIFICATES AND PRIVATE KEYS



CERTIFICATE (.CRT / .PEM)
Identifies your domain.

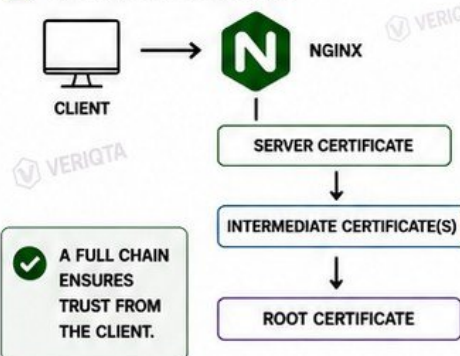


PRIVATE KEY (.KEY)
Proves ownership of the certificate.



KEEP PRIVATE KEYS SECURE AND RESTRICTED.

4. CERTIFICATE CHAINS



5. CONFIGURING HTTPS IN NGINX

```
server {
    listen 443 ssl http2;
    server_name example.com www.example.com;

    ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;

    # Strong protocols
    ssl_protocols TLSv1.2 TLSv1.3;

    # Cipher configuration (modern and secure)
    ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:
    ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305';
    ssl_prefer_server_ciphers on;

    root /var/www/html;
}
```

6. TLS PROTOCOLS

- ✓ USE TLSv1.2 AND TLSv1.3 ONLY.
- ✓ DISABLE SSLv3, TLSv1.0, TLSv1.1.
- ✓ LATEST TLSv1.3 PROVIDES BETTER SECURITY AND PERFORMANCE.

7. CIPHER CONSIDERATIONS

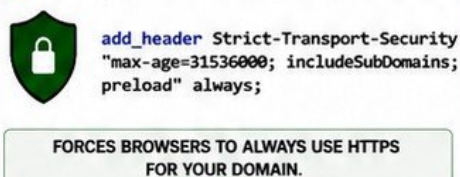
- ✓ USE STRONG, MODERN CIPHERS.
- ✓ PREFER ECDHE KEY EXCHANGE.
- ✓ ENABLE FORWARD SECRECY.
- ✓ AVOID WEAK OR DEPRECATED CIPHERS.

8. HTTP TO HTTPS REDIRECTS

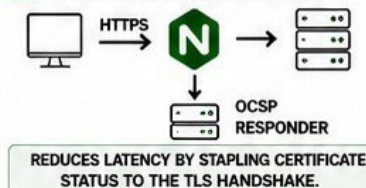
```
server {
    listen 80;
    server_name example.com www.example.com;

    return 301 https://$host$request_uri;
}
```

9. HSTS (HTTP STRICT TRANSPORT SECURITY)



10. OCSP STAPLING CONCEPTS



11. CERTIFICATE RENEWAL

- ✓ CERTIFICATES EXPIRE.
- ✓ MONITOR EXPIRATION DATES.
- ✓ AUTOMATE RENEWAL.
- ✓ TEST AFTER RENEWAL.
- ✓ AVOID SERVICE DISRUPTION.



12. LET'S ENCRYPT AND CERTBOT WORKFLOW



COMMON CERTBOT COMMANDS

```
sudo certbot --nginx
sudo certbot renew --dry-run
sudo certbot certificates
sudo certbot renew
```

13. TESTING TLS CONFIGURATION

- ✓ USE SSL LABS TO TEST YOUR SERVER.
- ✓ VERIFY PROTOCOLS AND CIPHERS.
- ✓ CHECK CERTIFICATE CHAIN.
- ✓ CONFIRM HSTS AND OCSP STAPLING.
- ✓ TEST FROM MULTIPLE NETWORKS.

TOOLS: SSL LABS, CURL, OPENSSL, BROWSER DEVTOOLS.

14. COMMON CERTIFICATE FAILURES

- ✗ EXPIRED CERTIFICATE
- ✗ MISMATCHED DOMAIN (CN / SAN)
- ✗ INCOMPLETE CERTIFICATE CHAIN
- ✗ USING SELF-SIGNED CERTIFICATES IN PRODUCTION
- ✗ WEAK CIPHERS OR OLD TLS PROTOCOLS
- ✗ INVALID PRIVATE KEY
- ✗ INCORRECT FILE PERMISSIONS
- ✗ MISSING INTERMEDIATE CERTIFICATES
- ✗ CERT NOT TRUSTED BY CLIENT
- ✗ FAILED DOMAIN VALIDATION



SECURE TLS CONFIGURATION BUILDS TRUST, PROTECTS DATA, AND IMPROVES SEO AND USER CONFIDENCE.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

VERIQTA — BUILDING SKILLS. SOLVING PROBLEMS. CREATING IMPACT.

NGINX SECURITY HARDENING



LOCK DOWN. PROTECT USERS. REDUCE RISK.



1. REMOVING UNNECESSARY INFORMATION DISCLOSURE

- Prevent NGINX from exposing version information.

```
http {
    server_tokens off;
}
```



3. CONTENT SECURITY POLICY (CSP) CONCEPTS

- Helps prevent XSS, data injection, and other attacks.

```
add_header Content-Security-Policy
"default-src 'self'; script-src 'self';
object-src 'none'; frame-ancestors 'self';"
always;
```



2. SECURITY HEADERS

- Add essential security headers to every response.

```
add_header Strict-Transport-Security "max-age=63072000;"
always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header Referrer-Policy "strict-origin-when-cross-origin"
always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Content-Security-Policy "default-src 'self';"
always;
```



4. X-CONTENT-TYPE-OPTIONS

- Prevent MIME-type sniffing.

```
add_header X-Content-Type-Options
"nosniff" always;
```



5. REFERRER POLICY

- Control how much referrer information is shared.

```
add_header Referrer-Policy
"strict-origin-when-cross-origin"
always;
```



6. PERMISSIONS

- Ensure proper file and directory permissions.

- NGINX user should have minimum access.
- Avoid writable directories in web root.
- Protect configuration files.



7. PROTECTING SENSITIVE FILES

- Block access to sensitive files and folders.

```
location ~* \.(env|ini|log|sh|sql|conf)$ {
    deny all;
    return 403;
}
```



8. BLOCKING HIDDEN FILES

- Prevent access to hidden files like .git, .env, etc.

```
location ~* /\. {
    deny all;
    return 403;
}
```



9. REQUEST METHOD RESTRICTIONS

- Allow only the necessary HTTP methods.

```
location /api/ {
    limit_except GET POST {
        deny all;
    }
}
```



10. IP ALLOWLISTS AND DENYLISTS

- Control access by IP address.

```
location /admin/ {
    allow 192.168.1.0/24;
    allow 10.0.0.0/8;
    deny all;
}
```



11. BASIC AUTHENTICATION

- Protect admin or sensitive areas.

```
location /admin/ {
    auth_basic "Restricted Area";
    auth_basic_user_file /etc/nginx/.htpasswd;
}
```



12. LIMITING ATTACK SURFACE

- Disable unnecessary modules.
- Close unused ports.
- Keep NGINX and system updated.
- Isolate services.
- Use firewalls.



13. PRODUCTION HARDENING CHECKLIST

- server_tokens off;
- Use HTTPS with strong TLS.
- Set critical security headers.
- Protect sensitive files and directories.
- Block hidden files.
- Restrict HTTP methods.
- Configure IP allowlists / denylists.
- Use authentication for sensitive areas.
- Run NGINX with least privileges.
- Monitor logs and suspicious activity.
- Keep NGINX and system updated.



SECURITY IS A LAYERED APPROACH.

CONFIGURE NGINX SECURELY, MONITOR CONTINUOUSLY, AND KEEP IMPROVING.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



RATE LIMITING AND CONNECTION PROTECTION

CONTROL TRAFFIC. PROTECT RESOURCES. PREVENT ABUSE.



1. WHY RATE LIMITING MATTERS

- Prevents abuse, brute force, and scraping.
- Protects backend servers and databases.
- Maintains availability during traffic spikes.
- Ensures fair usage for all legitimate users.
- Reduces cost and resource exhaustion.



2. RATE LIMITING BASICS

- **limit_req_zone**: Define a shared memory zone to track request rates.
- **limit_req**: Apply rate limiting to a location.



3. LIMIT_REQ_ZONE

Define a zone to store client request rates.

```
http {
    limit_req_zone $binary_remote_addr
    zone=req_limit_per_ip:10m rate=10r/s;
}
```

- **\$binary_remote_addr**: Unique client IP key.
- **zone=req_limit_per_ip:10m**: Zone name and size.
- **rate=10r/s**: Allow 10 requests per second.



4. LIMIT_REQ

Apply rate limiting to a location.

```
location /api/ {
    limit_req zone=req_limit_per_ip
    burst=20 nodelay;
    proxy_pass http://backend;
}
```

- Applies the defined rate limit to this location.
- Rejects excess requests based on burst and nodelay.



5. REQUEST RATE AND BURST TRAFFIC

- Request rate: Average allowed requests per second.
- Burst traffic: Short spikes above the rate.
- Burst helps absorb short spikes without rejection.

6. BURST

Allows extra requests beyond the rate.

```
burst=20;
```

Allows up to 20 extra requests in a short time.

7. NODELAY

Process excess requests immediately instead of queuing them.

```
nodelay;
```

Rejects excess requests immediately (no delay).



8. CONNECTION LIMITING

Limits the number of concurrent connections from a client.

- **limit_conn_zone**: Define a shared memory zone.
- **limit_conn**: Apply connection limits.

9. LIMIT_CONN_ZONE

```
http {
    limit_conn_zone
    $binary_remote_addr
    zone=conn_limit_per_ip:10m;
}
```

10. LIMIT_CONN

```
location / {
    limit_conn
    conn_limit_per_ip 10;
}
```

Allows max 10 concurrent connections per IP.



11. PROTECTING LOGIN ENDPOINTS

- Prevent brute force attacks on login routes.

```
location /login/ {
    limit_req zone=req_limit_per_ip
    burst=5 nodelay;
    limit_conn conn_limit_per_ip 5;
    proxy_pass http://backend;
}
```



12. PROTECTING APIS

- Protect APIs from abuse and scraping.

```
location /api/ {
    limit_req zone=req_limit_per_ip
    burst=20 nodelay;
    limit_conn conn_limit_per_ip 20;
    proxy_pass http://backend;
}
```



13. PROTECTING EXPENSIVE APPLICATION ROUTES

- Limit expensive operations and heavy endpoints.

```
location /report/ {
    limit_req zone=req_limit_per_ip
    burst=10 nodelay;
    proxy_pass http://backend;
}
```



14. AVOID ACCIDENTAL BLOCKING OF LEGITIMATE TRAFFIC

- Set reasonable rates and bursts.
- Monitor before tightening limits.
- Allow trusted IPs if needed.
- Use separate limits for different user groups.



15. TESTING RATE LIMITS

- Use tools to simulate requests and verify behavior.
- Test normal traffic, bursts, and limits.
- Monitor NGINX logs and status codes.
- Adjust rates and bursts based on real usage.

```
# Test normal rate
ab -n 100 -c 1 http://yourdomain.com/api/
# Test burst traffic
ab -n 100 -c 50 http://yourdomain.com/api/
# Watch logs
tail -f /var/log/nginx/error.log
```

COMMON RATE LIMIT RESPONSES

```
200 OK
429 Too Many Requests
503 Service Unavailable
(when limits are strict)
```



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

11. CACHING AND STATIC CONTENT PERFORMANCE



1. STATIC FILE SERVING

- NGINX serves static files directly from disk.
- Fast, efficient, and low resource usage.
- Ideal for images, CSS, JS, fonts, videos.



2. BROWSER CACHING

- Store static assets in the user's browser.
- Reduces server load and improves speed.



3. EXPIRES DIRECTIVE

- Sets how long the browser should cache a resource.
- Example:

```
location ~* \.(jpg|png|css|js)$ {
    expires 30d;
}
```



4. CACHE-CONTROL HEADER

- More flexible and powerful than expires.
- Example:

```
add_header Cache-Control
    "public, max-age=2592000";
# 30 days
```



13. AVOIDING CACHE STAMPEDES

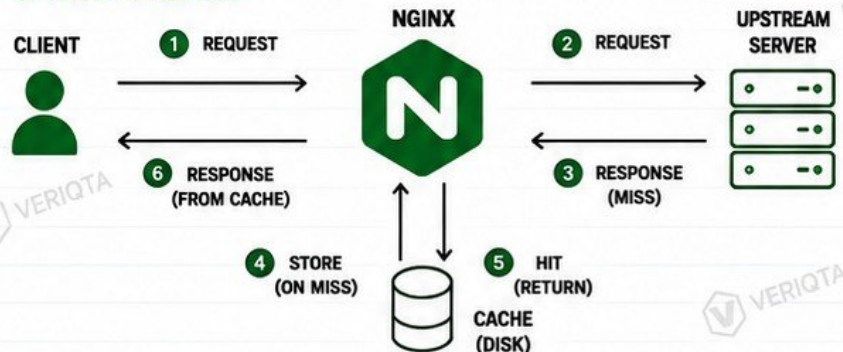
- Enable cache locking.
- Use stale-while-revalidate.
- Add jitter to expiration times if needed.



PRO TIP

Good caching reduces latency, improves user experience, and lowers infrastructure costs. Cache smart, invalidate properly, and monitor continuously.

5. PROXY CACHING



6. PROXY_CACHE_PATH

- Defines where and how cache is stored.
- Example:

```
proxy_cache_path /var/cache/nginx levels=1:2
    keys_zone=my_cache:100m max_size=10g
    inactive=60m use_temp_path=off;
```



7. CACHE KEYS

- Unique identifier for cached content.
- Default: \$scheme\$proxy_host\$request_uri
- Can customize using proxy_cache_key.



8. CACHE VALIDITY

- Controls how long content stays in cache.
- Set with proxy_cache_valid.
- Example:

```
proxy_cache_valid 200 302 10m;
proxy_cache_valid 404 1m;
```



9. CACHE BYPASS

- Bypass cache for specific requests.
- Use map + proxy_no_cache.
- Common for logged-in users or query strings.



10. CACHE HITS & MISSES

- HIT: Served from cache.
- MISS: Fetched from upstream and stored.
- Check \$upstream_cache_status in logs (HIT, MISS, BYPASS, EXPIRED, STALE).



11. STALE CONTENT

- Serve expired content while updating in background.
- Improves availability.
- Example:

```
proxy_cache_use_stale error timeout updating;
```



12. CACHE LOCKING

- Prevents multiple requests from updating the same cache key.
- Use proxy_cache_lock on;
- Avoids thundering herd during cache refresh.



14. WHEN NOT TO CACHE

- Highly dynamic content.
- User-specific content.
- Sensitive or personalized data.
- Real-time financial or stock data.
- Frequently changing APIs.



15. TROUBLESHOOTING INCORRECT CACHED RESPONSES

- Check cache key.
- Verify headers (Cache-Control, Expires, Vary).
- Review cache status in logs.
- Clear cache for specific keys.
- Ensure upstream sends correct headers.



USEFUL VARIABLES

- \$upstream_cache_status
- \$upstream_cache_time
- \$proxy_cache_key



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

12. COMPRESSION, BUFFERING, AND PERFORMANCE



1. GZIP COMPRESSION

- Reduces response size before sending to client.
- Saves bandwidth and speeds up page load.

```
gzip on;  
gzip_types text/plain text/css  
application/json application/javascript  
text/xml application/xml application/xml+rss  
text/javascript image/svg+xml;  
gzip_comp_level 5;  
gzip_min_length 1024;  
vary on;
```



2. COMPRESSION TYPES

- Compress only text-based content.

COMMON TYPES TO COMPRESS:

- text/*
- application/json
- application/javascript
- application/xml
- text/css
- image/svg+xml



3. COMPRESSION LEVELS

- Level 1 = Fastest, least CPU usage, larger size.
- Level 5 = Balanced (recommended).
- Level 9 = Best compression, highest CPU usage.

LEVEL	COMPRESSION	CPU USAGE
1	LOW	LOW
5	MEDIUM (RECOMMENDED)	MEDIUM
9	HIGH	HIGH



4. REQUEST BUFFERING

- Buffers client request body before sending to upstream.
- Protects upstream from slow clients.

```
client_body_buffer_size 128k;  
client_body_buffering on;  
client_max_body_size 50m;
```



5. RESPONSE BUFFERING

- Buffers responses from upstream before sending to client.
- Improves performance for slow clients.

```
proxy_buffering on;  
proxy_buffers 16 16k;  
proxy_buffer_size 32k;  
proxy_busy_buffers_size 64k;
```



6. PROXY BUFFERS

- Temporary memory for storing upstream responses.
- Helps handle large or slow responses efficiently.

```
proxy_buffers 16 16k; # Number  
proxy_buffer_size 32k; # First buffer  
proxy_busy_buffers_size 64k;
```



7. LARGE RESPONSES

- Use buffering and temp files for large responses.
- Prevents memory exhaustion.

```
proxy_max_temp_file_size 1024m;  
proxy_temp_file_write_size 64k;  
proxy_buffering on;
```



8. CLIENT BODY BUFFERING

- Buffers client upload data to disk or memory before proxying.
- Important for large file uploads.

```
client_body_buffer_size 128k;  
client_body_temp_path /var/lib/nginx/body;  
client_body_buffering on;
```



9. SENDFILE

- Sends files directly from disk to network.
- Reduces CPU usage and improves throughput.

```
sendfile on;  
tcp_nopush on;
```



10. TCP OPTIMIZATIONS

- Fine-tune TCP for better network performance.

```
tcp_nodelay on;  
tcp_nodelay on;  
tcp_nodelay on;
```



11. KEEPAIVE

- Keep connections open for multiple requests.
- Reduces handshake overhead.

```
keepalive_timeout 65;  
keepalive_requests 1000;
```



12. CONNECTION REUSE

- Reuse upstream connections with keepalive.
- Improves performance and reduces latency.

```
upstream backend {  
    server app1:8080;  
    server app2:8080;  
    keepalive 32;  
}
```



13. TIMEOUT TUNING

- Set timeouts based on application behavior and network conditions.

```
proxy_connect_timeout 5s;  
proxy_send_timeout 60s;  
proxy_read_timeout 60s;  
send_timeout 60s;
```



14. PERFORMANCE VS RESOURCE CONSUMPTION

- More buffering = More memory usage.
- Higher compression = More CPU usage.
- More connections = More memory and file descriptors.
- Balance performance, cost, and stability.
- Monitor CPU, memory, and network before changing settings.



15. AVOIDING PREMATURE TUNING

- Default settings work well for most workloads.
- Tune based on real metrics, not guesswork.
- Identify bottlenecks first (CPU, I/O, network, or upstream).
- Change one setting at a time and measure.
- Document changes and review regularly.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



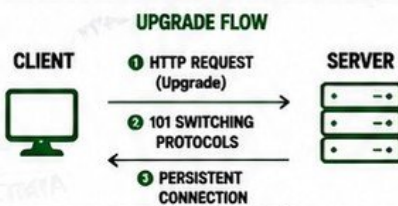
Telegram
veriqta

13. WEBSOCKETS AND LONG-LIVED CONNECTIONS



1. HTTP UPGRADE MECHANISM

- WebSockets start as a normal HTTP request.
- The client requests an upgrade to the WebSocket protocol.
- The server switches protocols and keeps the connection open.



2. WEBSOCKET PROXYING

- NGINX must pass the upgrade headers to the upstream server.
- Use HTTP/1.1 and the correct headers.

```

location /ws/ {
    proxy_pass http://backend;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_read_timeout 86400s; # Long-lived
    proxy_send_timeout 86400s;
}
    
```

3. UPGRADE AND CONNECTION HEADERS

- Upgrade: websocket
 - Tells the server to switch protocols.
- Connection: upgrade
 - Tells the server this is a special upgrade connection.

REQUIRED HEADERS

Upgrade: websocket
Connection: upgrade



4. HTTP/1.1 REQUIREMENTS

- WebSockets require HTTP/1.1.
- HTTP/2 does not support WebSockets directly.
- Ensure proxy_http_version 1.1;
- Ensure headers are passed correctly.



5. LONG-LIVED CONNECTIONS

- WebSockets keep the connection open for minutes, hours, or days.
- Ideal for real-time features like chat, notifications, dashboards, and gaming.
- Requires proper timeout tuning.



6. PROXY READ TIMEOUT

- Default proxy_read_timeout is 60s.
- Too low = connection closed by NGINX.
- Set high for WebSockets.
- Example: 86400s (24 hours)

```
proxy_read_timeout 86400s;
```



7. IDLE CONNECTIONS

- Some clients or networks drop idle connections.
- Use ping/pong at the application level.
- Increase timeouts to avoid disconnects.
- Example:

```

proxy_read_timeout 86400s;
proxy_send_timeout 86400s;
    
```



8. WEBSOCKET LOAD BALANCING

- Use upstream block with multiple servers.
- Ensure sticky sessions if needed.
- IP hash can help for session affinity.
- Example:

```

upstream ws_backend {
    ip_hash;
    server 10.0.0.1:8080;
    server 10.0.0.2:8080;
    keepalive 64;
}
    
```



9. COMMON WEBSOCKET FAILURES

- Connection closed unexpectedly.
- 400 Bad Request
- 502 Bad Gateway
- Idle timeout disconnects.
- Network interruptions.
- Missing or incorrect headers.



10. 400 AND 502 ERRORS

400 BAD REQUEST

- Missing or incorrect Upgrade header.
- Missing or incorrect Connection header.
- HTTP/1.0 used instead of HTTP/1.1.
- Invalid WebSocket handshake.

502 BAD GATEWAY

- Upstream server not running.
- Upstream closed connection.
- Timeout from upstream.
- Incorrect proxy configuration.



11. DEBUGGING DROPPED CONNECTIONS

- Check error logs for disconnect reasons.
- Increase log level temporarily.
- Use browser dev tools → Network → WS.
- Check upstream server logs.
- Use TCP dump if needed.
- Verify timeouts, headers, and network path.
- Example commands:

```

tail -f /var/log/nginx/error.log
tcpdump -i any port 443 -n
ss -tnp | grep ESTAB
    
```



12. PRODUCTION WEBSOCKET CONFIGURATION CHECKLIST

- ✓ Use proxy_http_version 1.1;
- ✓ Set Upgrade and Connection headers.
- ✓ Increase proxy_read_timeout.
- ✓ Increase proxy_send_timeout.
- ✓ Use keepalive for upstream.
- ✓ Load balance with session affinity if needed.
- ✓ Monitor error logs and connection drops.
- ✓ Test under real traffic conditions.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

14. NGINX LOGGING AND OBSERVABILITY



1. ACCESS LOGS

- Records every client request.
- Essential for analytics, auditing, and troubleshooting.
- Default: `/var/log/nginx/access.log`

```
127.0.0.1 - - [10/May/2025:10:15:23
+0000] "GET /index.html HTTP/1.1"
200 1024 "https://example.com/"
"Mozilla/5.0"
```



2. ERROR LOGS

- Records errors, warnings, and critical issues.
- Default: `/var/log/nginx/error.log`

```
2025/05/10 10:15:23 [error] 1234#1234:
*5678 connect() failed (111:
Connection refused) while connecting
to upstream, client: 127.0.0.1,
server: example.com, request: "GET /"
upstream: "http://10.0.0.5:8080/"
```



3. LOG LEVELS

- Control the verbosity of error logging.
- Levels (low to high severity):
debug, info, notice, warn, error, crit, alert, emerg
- Recommendation: warn or error in production.

```
error_log /var/log/nginx/error.log warn;
```



4. CUSTOM LOG_FORMAT

- Define custom log_format to log the fields that matter.
- Example: production-friendly format

```
log_format main '$remote_addr - $request_id [$time_local] "$request"
$status $body_bytes_sent "$http_referer" "$http_user_agent"
$request_time $upstream_response_time $upstream_addr
$http_x_forwarded_for $host';
```

FIELD EXPLANATION

FIELD	DESCRIPTION
<code>\$remote_addr</code>	Client IP address
<code>\$request_id</code>	Unique request ID (useful for tracing)
<code>\$request</code>	HTTP request line
<code>\$status</code>	HTTP status code
<code>\$body_bytes_sent</code>	Response size in bytes
<code>\$request_time</code>	Total time NGINX took to process the request
<code>\$upstream_response_time</code>	Time taken by upstream server
<code>\$upstream_addr</code>	Address of the upstream server
<code>\$http_x_forwarded_for</code>	Original client IP (behind proxy/load balancer)
<code>\$host</code>	Requested host



5. WHAT TO LOG (KEY DATA POINTS)

- ✓ Request time (`$request_time`)
- ✓ Upstream response time (`$upstream_response_time`)
- ✓ Upstream address (`$upstream_addr`)
- ✓ HTTP status (`$status`)
- ✓ Client IP (`$remote_addr`, `$http_x_forwarded_for`)
- ✓ Request ID (`$request_id`)
- ✓ Bytes sent (`$body_bytes_sent`)
- ✓ HTTP method and URI (`$request`)
- ✓ User agent (`$http_user_agent`)
- ✓ Referrer (`$http_referer`)



6. STRUCTURED LOGGING

- Use JSON logs for better parsing and analysis.
- Example JSON log format:

```
log_format json_log escape=json '{
  "time": "$time_iso8601",
  "request_id": "$request_id",
  "remote_addr": "$remote_addr",
  "request": "$request",
  "status": "$status",
  "body_bytes_sent": "$body_bytes_sent",
  "request_time": "$request_time",
  "upstream_response_time": "$upstream_response_time",
  "upstream_addr": "$upstream_addr",
  "host": "$host",
  "user_agent": "$http_user_agent"
}';
```

```
access_log /var/log/nginx/access.json json_log;
```



7. LOG ROTATION

- Prevents log files from growing indefinitely.
- Use logrotate (recommended).
- Example: `/etc/logrotate.d/nginx`

```
/var/log/nginx/*.log {
  daily
  rotate 14
  compress
  delaycompress
  missingok
  notifempty
  create 0640 www-data adm
  sharedscripts
  postrotate
    systemctl reload nginx > /dev/null
  endscript
}
```



8. CENTRALIZED LOGGING

- Send logs to a central system for storage and analysis.
- Common tools: ELK Stack, Grafana Loki, Graylog, Splunk.
- Use syslog or log shipper (Filebeat, Fluent Bit).



9. METRICS

- Metrics help you understand performance and capacity.
- Key metrics:
 - ✓ Requests per second (RPS)
 - ✓ Active connections
 - ✓ 4xx / 5xx error rates
 - ✓ Upstream response time
 - ✓ Connection states
 - ✓ Cache hit rate (if using proxy_cache)



10. PROMETHEUS MONITORING APPROACHES

- NGINX does not expose metrics by default.
- Options:
 - ✓ NGINX Plus API (built-in)
 - ✓ NGINX VTS Module
 - ✓ Exporter (nginx-prometheus-exporter)
 - ✓ Ingress Controller metrics (Kubernetes)
- Example Prometheus scrape (nginx exporter):

```
- job_name: 'nginx'
  static_configs:
    - targets: ['127.0.0.1:9113']
```



11. BUILDING USEFUL PRODUCTION VISIBILITY

- ✓ Correlate logs with metrics.
- ✓ Track latency at every layer.
- ✓ Alert on high error rates and latency.
- ✓ Use dashboards for real-time health.
- ✓ Include request IDs in logs and traces.
- ✓ Monitor both NGINX and upstream systems.
- ✓ Review logs regularly and automate alerts.



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

15. DEBUGGING NGINX HTTP ERRORS

COMMON NGINX HTTP ERRORS

ERROR	NAME	MEANING	COMMON CAUSES	WHAT TO CHECK
400	Bad Request	NGINX cannot understand the client request.	- Malformed request - Invalid headers - Large or broken headers	- Client request syntax - Headers and URL - Proxy buffer settings
401	Unauthorized	Authentication is required and has failed or not provided.	- Missing or invalid credentials - Auth module configured	- Auth configuration - Authorization headers - Upstream auth setup
403	Forbidden	Access is understood but refused.	- Permissions - IP restrictions - Deny rules - Authz failures	- Location and server rules - IP allow/deny - File permissions
404	Not Found	NGINX cannot find the requested resource.	- Wrong URI - Missing file - Incorrect root/alias	- Request URI - root / alias paths - File existence
408	Request Timeout	Client took too long to send the request.	- Slow client - Client header timeout	- client_header_timeout - Network latency - Client behavior
413	Payload Too Large	Request body is too large.	- Body exceeds limit - Large file upload	- client_max_body_size - proxy_request_buffering - Upstream limits
429	Too Many Requests	Rate limit exceeded.	- limit_req triggered - Application rate limit	- limit_req configuration - Burst settings - Client behavior
499	Client Closed Request	Client closed the connection before the response.	- User closed browser - Network interruption - Long upstream processing	- Access logs (499) - Upstream response time - Client stability
500	Internal Server Error	Unexpected condition prevented fulfillment of the request.	- NGINX configuration error - Upstream application error - Script or module failure	- error.log - Upstream logs - Configuration
502	Bad Gateway	NGINX received an invalid response from upstream.	- Upstream down - Invalid response - Connection reset	- Upstream health - Network path - error.log
503	Service Unavailable	Server is not ready to handle the request.	- Upstream unavailable - Maintenance mode - Resource exhaustion	- Upstream status - System resources - Graceful shutdowns
504	Gateway Timeout	Upstream did not respond in time.	- Upstream slow - Network latency - Timeout too low	- proxy_read_timeout - Upstream performance - Network issues

ERROR-TO-LAYER MAPPING

ERROR	LIKELY LAYER	EXAMPLE ISSUE
400	CLIENT / REQUEST	Malformed request
401	AUTHENTICATION	Missing credentials
403	SECURITY / ACCESS	Access denied
404	ROUTING / APP	Resource not found
408	CLIENT / NETWORK	Slow request sender
413	CLIENT / APP	Large request body
429	APPLICATION RATE LIMIT	Too many requests
499	CLIENT / NETWORK	Client closed connection
500	NGINX / APPLICATION	Internal failure
502	UPSTREAM / NETWORK	Bad upstream response
503	UPSTREAM / SYSTEM	Service not available
504	UPSTREAM / NETWORK	Upstream timeout

LOGS TO INSPECT

1. ACCESS LOG

- Contains status codes (e.g., 502, 499)
- Shows request details and timings
- Helps identify client behavior

Example:

```
127.0.0.1 -- [10/May/2025:10:15:23 +0000]  
"GET /api/data HTTP/1.1" 502 157 "-"  
"Mozilla/5.0" upstream_time=2.504
```

2. ERROR LOG

- Shows root causes and failures
- Upstream errors
- Timeouts
- Configuration issues

Example:

```
2025/05/10 10:15:23 [error] 1233#1234:  
*5678 upstream timed out (110: Connection  
timed out) while reading response header  
from upstream, client: 127.0.0.1,  
server: example.com, request: "GET /  
api/data HTTP/1.1", upstream: "http://  
10.0.0.5:8080/api/data"
```

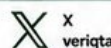
TROUBLESHOOTING WORKFLOW

- IDENTIFY THE ERROR CODE**
Check access logs and reproduce the issue.
- MAP THE ERROR TO THE LAYER**
Use the error-to-layer mapping table.
- CHECK ACCESS LOGS**
Review status code, request details, upstream time, and client info.
- CHECK ERROR LOGS**
Look for specific NGINX or upstream error messages.
- VALIDATE UPSTREAM HEALTH**
Confirm upstream is up and responding.
- VERIFY CONFIGURATION**
Check relevant NGINX directives, timeouts, and limits.
- TEST AND REPRODUCE**
Use curl or browser to test from client and server sides.
- FIX ROOT CAUSE**
Apply fix, reload NGINX, and monitor.



QUICK TIPS

- ✓ Always check both access and error logs.
- ✓ Use upstream_response_time and upstream_addr in access logs.
- ✓ Monitor 499, 502, 503, 504 carefully.
- ✓ Set appropriate timeouts.
- ✓ Test under real production conditions.
- ✓ Do not guess—follow the workflow.
- ✓ Log upstream health continuously.
- ✓ Correlate logs with metrics.
- ✓ Document common issues and fixes.



TROUBLESHOOTING 502 AND 504 INCIDENTS

METHODICAL INVESTIGATION = FASTER ROOT CAUSE + QUICKER RECOVERY

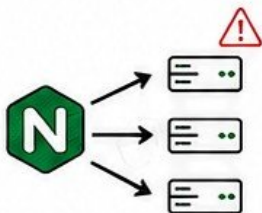
SCENARIO 1

NGINX RETURNS **502 BAD GATEWAY** AFTER AN APPLICATION DEPLOYMENT



INVESTIGATE

- Backend process
- Upstream address
- Application port
- Unix sockets
- DNS
- Connection refusal
- Application crashes
- Logs



STEP-BY-STEP INVESTIGATION PATH

1 CONFIRM THE ERROR

```
curl -I https://yourdomain.com
# Expect: HTTP/1.1 502 Bad Gateway
```

2 CHECK NGINX ERROR LOG

```
sudo tail -f /var/log/nginx/error.log
```

3 CHECK UPSTREAM STATUS

```
sudo nginx -T | grep -A 20 upstream
```

4 VERIFY BACKEND PROCESS IS RUNNING

```
ps aux | grep your_app
systemctl status your_app
```

5 CHECK BACKEND PORT LISTENING

```
ss -tulpn | grep :<port>
netstat -tulpn | grep :<port>
```

6 TEST CONNECTIVITY TO BACKEND

```
curl -v http://127.0.0.1:<port>/health
curl -v http://<backend-ip>:<port>/health
```

7 IF USING UNIX SOCKET

```
ls -l /path/to/socket.sock
curl --unix-socket /path/to/socket.sock
http://localhost/health
```

8 CHECK DNS RESOLUTION (IF USING HOSTNAMES)

```
dig <backend-hostname>
nslookup <backend-hostname>
```

9 LOOK FOR APPLICATION CRASHES

```
journalctl -u your_app --since "10 min ago"
dmesg | tail -20
```

10 RELOAD NGINX AFTER FIX (NOT RESTART)

```
sudo nginx -t && sudo systemctl reload nginx
```

SCENARIO 2

USERS RECEIVE **504 GATEWAY TIMEOUT** ONLY DURING PEAK TRAFFIC



INVESTIGATE

- Application latency
- Database latency
- Upstream timeouts
- Worker saturation
- Connection pools
- Backend capacity
- Dependency failures



STEP-BY-STEP INVESTIGATION PATH

1 CONFIRM THE ERROR UNDER LOAD

```
curl -I https://yourdomain.com
# Expect: HTTP/1.1 504 Gateway Timeout
```

2 CHECK ERROR LOG FOR TIMEOUTS

```
sudo tail -f /var/log/nginx/error.log
# Look for "upstream timed out"
```

3 CHECK UPSTREAM RESPONSE TIME

```
grep 'upstream_response_time' /var/log/nginx/access.log |
awk '{print $NF}' | sort -nr | head
```

4 MONITOR BACKEND LATENCY

```
curl -w "\nTotal Time: %{time_total}s\n"
http://<backend-ip>:<port>/health
```

5 CHECK DATABASE LATENCY

```
mysqladmin -u user -p status
# or use BB monitoring tools
```

6 CHECK NGINX WORKER SATURATION

```
ps aux | grep nginx
cat /proc/$(pgrep -n nginx)/status | grep "Voluntary\|
Nonvoluntary"
```

7 CHECK ACTIVE CONNECTIONS

```
ss -s
ss -tanp | grep :<backend-port> | wc -l
```

8 CHECK UPSTREAM TIMEOUT CONFIGURATION

```
nginx -T | grep -E "proxy_connect_timeout\|
proxy_read_timeout\|proxy_send_timeout" -n
```

9 CHECK BACKEND CAPACITY

```
top -b -n 1 | head -20
free -h
df -h
```

10 CHECK DEPENDENCIES (CACHE, QUEUE, API, ETC.)

```
curl -I https://dependency-service/health
redis-cli ping # Example for Redis
```

QUICK REFERENCE COMMANDS

NGINX TEST & RELOAD

- nginx -t
- nginx -T
- systemctl reload nginx
- systemctl status nginx

LOGS

- tail -f /var/log/nginx/access.log
- tail -f /var/log/nginx/error.log
- journalctl -u nginx -f

NETWORK & CONNECTIONS

- ss -tulpn
- ss -tanp
- netstat -tulpn
- dig domain.com

UPSTREAM TEST

- curl -v http://127.0.0.1:port
- curl -v http://<ip>:<port>
- curl --unix-socket /path/sock.sock
http://localhost/health



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

ZERO-DOWNTIME DEPLOYMENTS AND TRAFFIC CONTROL

KEEP SERVICES AVAILABLE • CONTROL TRAFFIC • DEPLOY SAFELY

1 GRACEFUL CONFIGURATION RELOAD



- Reload without dropping connections
- Existing connections continue
- New connections use new configuration

```
sudo nginx -t &&
sudo systemctl reload nginx
```

2 TESTING BEFORE RELOAD



- Always test configuration
- Catch syntax errors
- Validate changes before applying

```
sudo nginx -t
```

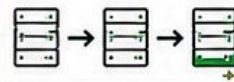
3 REMOVING BACKENDS SAFELY



- Mark backend down before removing
- Stop new connections
- Let existing connections finish

```
server backup.example.com
max_fails=0;
down;
```

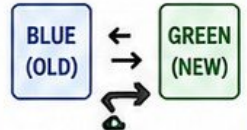
4 ROLLING DEPLOYMENTS



- Update instances one at a time
- Keep others serving traffic
- Zero downtime

Update -> Drain -> Remove
-> Deploy -> Restore

5 BLUE-GREEN DEPLOYMENTS



- Two identical environments
- Switch traffic instantly
- Quick rollback by switching back

Canary -> Monitor -> Increase
-> Full Rollout

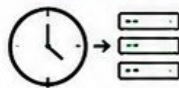
7 WEIGHTED TRAFFIC DISTRIBUTION



- Control traffic share between versions
- Useful for canary and gradual rollouts

weight=80 (old)
weight=20 (new)

8 CONNECTION DRAINING CONCEPTS



- Stop accepting new connections
- Allow existing connections to complete
- Prevent dropped requests

Use 'down' or remove backend
after drain timeout

9 DEPLOYMENT VALIDATION



- Check application health
- Validate logs and metrics
- Confirm user experience
- Automated and manual checks

Health endpoints, synthetic
checks, logs, metrics

10 FAILED RELEASE HANDLING



- Detect issues quickly
- Stop rollout immediately
- Protect users
- Communicate and mitigate

Alert -> Stop -> Rollback
-> Investigate

11 ROLLBACK



- Revert to last known good version
- Fast and reliable
- Part of every plan

Switch back / Restore
previous config

12 AVOIDING DROPPED CONNECTIONS



- Use graceful reloads
- Drain connections
- Keep timeouts appropriate
- Monitor during deploys

reload -> drain -> deploy
-> validate

PRODUCTION DEPLOYMENT WORKFLOW

1 PLAN



Define changes
and rollback plan

2 TEST CONFIG



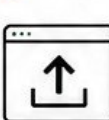
Validate with
nginx -t

3 PREPARE



Ensure backends
are healthy

4 DEPLOY



Reload or update
in stages

5 VALIDATE



Check health, logs,
metrics, users

6 MONITOR



Monitor closely
for issues

7 COMPLETE



Confirm success
and document



**ZERO-DOWNTIME IS A PROCESS, NOT A SINGLE COMMAND.
TEST, DEPLOY GRADUALLY, MONITOR, AND ROLLBACK FAST IF NEEDED.**



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

NGINX WITH DOCKER AND KUBERNETES

MODERN DEPLOYMENT • SCALABLE • CLOUD NATIVE • PRODUCTION READY

1 RUNNING NGINX IN CONTAINERS



- Lightweight and portable
- Consistent across environments
- Easy scaling and updates
- Ideal for microservices architectures

Docker Run Example

```
docker run -d --name nginx \
-p 80:80 \
-v $(pwd)/nginx.conf:/etc/nginx/nginx.conf:ro \
-v $(pwd)/html:/usr/share/nginx/html:ro \
nginx:alpine
```

2 CONTAINER CONFIGURATION



- nginx.conf is the main file
- Custom includes in conf.d/
- Tune worker processes and connections
- Optimize timeouts, buffers, and keepalive

Minimal nginx.conf Example

```
worker_processes auto;
events {
    worker_connections 1024;
}
http {
    include /etc/nginx/conf.d/*.conf;
}
```

3 VOLUMES



- Persist configuration, logs, TLS certificates, and content
- Use bind mounts or named volumes
- Keep containers stateless

Volume Example

```
-v $(pwd)/logs:/var/log/nginx
-v $(pwd)/certs:/etc/nginx/certs
-v $(pwd)/conf:/etc/nginx/conf.d
```

4 CONFIGURATION INJECTION



- Inject configs using volumes
- Use environment variables with templates
- ConfigMaps (Kubernetes)
- Immutable container images

Env Variable Example

```
docker run -d -e NGINX_HOST=example.com \
-e NGINX_PORT=80 \
nginx:alpine
```

5 REVERSE PROXY BETWEEN CONTAINERS



CLIENT



- NGINX routes traffic to backend containers
- Service discovery via Docker DNS
- Load balance multiple application containers

Upstream Example

```
upstream app {
    server app1:3000;
    server app2:3000;
    server app3:3000;
}
```

6 DOCKER NETWORKING



USER-DEFINED
BRIDGE NETWORK

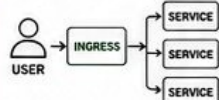
- Create user-defined networks
- Containers resolve by name
- Isolated and secure communication
- Expose only what is necessary

Create Network & Run

```
docker network create frontend
docker run --name nginx --network frontend -p 80:80 nginx
docker run --name app1 --network frontend app:latest
```

NGINX IN KUBERNETES

7 KUBERNETES INGRESS CONCEPTS



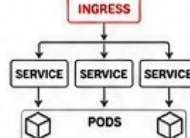
- HTTP/HTTPS routing layer
- Single entry point to cluster
- Rule-based traffic routing
- Works at Layer 7 (HTTP)

8 NGINX INGRESS CONTROLLER



- Popular and production-ready
- High performance
- Advanced annotations
- Active community support

9 SERVICES AND UPSTREAM WORKLOADS



- Ingress routes to Services
- Services load balance Pods
- Backend endpoints change dynamically

10 TLS AT INGRESS



- Terminate TLS at Ingress
- Use Kubernetes Secrets
- Supports Let's Encrypt
- End-to-end encryption supported

11 ROUTING RULES

/api	→ SERVICE A
/web	→ SERVICE B
*.example.com	→ SERVICE C

- Path-based routing
- Host-based routing
- Regex and rewrite rules
- Traffic splitting (canary)

12 LOGS AND TROUBLESHOOTING



- Check Ingress Controller logs
- kubectl logs -n ingress-nginx
- Use kubectl describe ingress
- Verify Endpoints and Service
- Check events and annotations

13 NGINX REVERSE PROXY VS KUBERNETES INGRESS

NGINX REVERSE PROXY (STANDALONE)	KUBERNETES INGRESS (WITH NGINX INGRESS CONTROLLER)
Runs outside Kubernetes	Runs inside Kubernetes
Manually managed and configured	Managed as Kubernetes resource
Static upstream servers or DNS	Dynamic upstream via Services
Best for traditional infrastructure	Best for cloud-native workloads
Full control over configuration	Easy integration with K8s features
Scaling requires manual setup	Auto-scaling and self-healing
Not aware of Kubernetes objects	Understands Pods, Services, Endpoints

14 USEFUL COMMANDS

DOCKER

```
docker ps
docker logs <nginx_container>
docker exec -it <nginx_container> sh
docker network ls
docker network inspect <network>
```

KUBERNETES

```
kubectl get ingress -A
kubectl describe ingress <name> -n <ns>
kubectl get svc -A
kubectl get endpoints -A
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller
kubectl get pods -n ingress-nginx
```



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

PRODUCTION INCIDENT INVESTIGATION LAB

A REAL-WORLD FRAMEWORK FOR FINDING ROOT CAUSE AND RESTORING STABILITY

INCIDENT



Users report intermittent API failures, but monitoring shows that NGINX itself is running.



USERS



NGINX



UPSTREAM
SERVERS

SYMPTOMS



- Some requests succeed
- Some return 502
- Latency increases during traffic spikes
- One backend behaves differently from the others

INVESTIGATION: STEP-BY-STEP PATH

1 CONFIRM CUSTOMER IMPACT



Check affected users, regions, and error rates.

```
curl -I https://api.example.com  
-H "X-Debug: incident"
```

2 INSPECT ACCESS LOGS



Look for 502 responses and response times.

```
grep " 502 " /var/log/nginx/  
access.log | tail -n 50
```

3 INSPECT ERROR LOGS



Find upstream errors, timeouts, and failures.

```
tail -n 100 /var/log/nginx/  
error.log
```

4 IDENTIFY AFFECTED UPSTREAM



Find which backend is returning errors.

```
grep "upstream" /var/log/nginx/  
error.log | tail -n 50
```

5 COMPARE UPSTREAM RESPONSE TIMES



Compare response time across all backends.

```
grep -oP 'upstream_response_time: \d{0-9}.*'  
/var/log/nginx/access.log | awk  
{sum+=$1} END{print "avg=", sum/NR}'
```

6 CHECK BACKEND HEALTH



Check health endpoints or application status.

```
curl -I http://backend1:8080/health  
curl -I http://backend2:8080/health
```

7 INSPECT CONNECTIONS



Check active connections to upstreams.

```
ss -s  
ss -tanp | grep :<backend-port>
```

8 VALIDATE DNS



Ensure DNS is resolving correctly and quickly.

```
dig +short backend1.example.com  
dig +short backend2.example.com
```

9 TEST INDIVIDUAL BACKEND SERVERS



Test each backend directly.

```
curl -I http://backend1:8080/api  
curl -I http://backend2:8080/api  
curl -I http://backend3:8080/api
```

10 REMOVE UNHEALTHY BACKEND



Temporarily remove unhealthy backend.

```
# Edit upstream block and comment out  
# the unhealthy server  
sudo nginx -t && sudo nginx -s reload
```

11 VERIFY RECOVERY



Monitor success rate and latency again.

```
watch -n 5 'curl -o /dev/null -s -w  
"%{http_code}%n" https://api.example.com'
```

12 RESTORE BACKEND SAFELY



Reintroduce backend after fixing issue.

```
# Re-enable server in upstream  
sudo nginx -t && sudo nginx -s reload
```

13 DETERMINE ROOT CAUSE



Analyze findings and logs to identify the real cause.

- Correlate logs, metrics, and backend behavior
- Check deployment changes
- Check application errors and database errors
- Identify resource exhaustion or dependency failures

```
journalctl -u your_app --since "30 min ago"  
dmesg | tail -n 100  
top -b -n 1 | head -n 30  
free -h
```

FINAL OUTCOME: DOCUMENT, REMEDIATE, PREVENT

ROOT CAUSE



Example:
One backend instance experienced high CPU usage due to a memory leak, causing slow responses and 502 errors.

IMMEDIATE REMEDIATION



- Removed unhealthy backend
- Traffic stabilized
- Error rate returned to normal
- Customer impact resolved
- Monitoring confirmed stability

LONG-TERM PREVENTION



- Fix memory leak in application
- Improve backend resource limits
- Add health checks
- Add alerts for latency spikes
- Improve deployment process

MONITORING IMPROVEMENTS



- Monitor 502/504 rates
- Monitor upstream latency
- Monitor backend health
- Monitor resource usage
- Alert on anomalies

QUICK COMMAND REFERENCE

LOGS

- tail -f /var/log/nginx/access.log
- tail -f /var/log/nginx/error.log
- grep " 502 " /var/log/nginx/access.log
- grep "upstream" /var/log/nginx/error.log

CONNECTIONS

- ss -s
- ss -tanp | grep :<port>
- netstat -tulnp | grep :<port>

HEALTH CHECKS

- curl -I http://backend:port/health
- curl -s http://backend:port/health
- nc -zv backend port

DNS

- dig +short domain.com
- dig domain.com
- nslookup domain.com



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



VERIQTA

PRODUCTION READINESS CHECKLIST AND COMMAND REFERENCE

1 CONFIGURATION

- ☒ Validate configuration before deployment
- ☒ Keep configuration version-controlled
- ☒ Use safe reloads
- ☒ Verify server and location matching
- ☒ Review proxy headers

2 RELIABILITY

- ☒ Configure upstream failure handling
- ☒ Set appropriate timeouts
- ☒ Protect against backend overload
- ☒ Test backend failures
- ☒ Test deployment rollback

3 SECURITY

- ☒ Enable HTTPS
- ☒ Protect sensitive files
- ☒ Configure appropriate security headers
- ☒ Restrict unnecessary access
- ☒ Apply rate limiting where appropriate

4 PERFORMANCE

- ☒ Review worker capacity
- ☒ Configure connection reuse
- ☒ Use caching where appropriate
- ☒ Compress suitable responses
- ☒ Monitor upstream latency

5 OBSERVABILITY

- ☒ Maintain useful access logs
- ☒ Monitor error rates
- ☒ Track upstream response times
- ☒ Monitor 499, 502, 503, and 504 responses
- ☒ Centralize production logs

ESSENTIAL COMMANDS

>_ `nginx -t`
Test configuration

>_ `nginx -T`
Dump full configuration

>_ `nginx -V`
Show version and modules

>_ `systemctl status nginx`
Check service status

>_ `systemctl reload nginx`
Reload configuration safely

>_ `systemctl restart nginx`
Restart NGINX service

>_ `journalctl -u nginx`
View systemd logs

>_ `tail -f /var/log/nginx/access.log`
Follow access log

>_ `tail -f /var/log/nginx/error.log`
Follow error log

>_ `curl -I https://example.com`
HTTP header request

>_ `curl -v https://example.com`
Verbose request for debugging

NGINX PRODUCTION TROUBLESHOOTING FLOW



USE THIS FLOW TO DETERMINE WHICH LAYER OWNS THE FAILURE BEFORE CHANGING CONFIGURATION.


[YouTube
veriqta](#)

[GitHub
veriqta](#)

[LinkedIn
veriqta](#)

[X
veriqta](#)

[Instagram
veriqta](#)

[Telegram
veriqta](#)